

In-class Exercise 3

● Graded

Student

Jinku Jung

Total Points

2 / 4 pts

Autograder Score

0.0 / 4.0

Question 2

Adjustment

■ 2 / 0 pts

+ 1 pt 1 point adjustment

✓ + 2 pts 2 point adjustment

+ 3 pts 3 point adjustment

+ 4 pts 4 point adjustment

✓ + 0 pts none

💬 This does not compile. There is something off with the brackets in Game.java Please be sure to check that all of your files compile before submitting.

Autograder Results

The autograder failed to execute correctly. Please ensure that your submission is valid. Contact your course staff for help in debugging this issue. Make sure to include a link to this page so that they can help you most effectively.

Submitted Files

```
1  //*****
2  // This class models a n-sided die
3  // it's the sam as it was in lecture
4  // no need to modify it
5  //
6  //*****
7
8
9  public class Die {
10
11     private int n;
12     private int side;
13
14     public Die(int shape){
15         n=shape;
16         roll();
17     }
18
19     public Die(){
20         n=6;
21         roll();
22     }
23
24     public void roll() {
25         side = (int) (Math.random() * n + 1);
26     }
27
28     public int getSide() {
29         return side;
30     }
31
32
33 } //end class
34
```

```
1  /*******
2  //   Example Test class simulating Pig computer vs. computer
3  //
4  // Your Game class must work with this class.
5  //
6  //
7  /*******
8
9
10
11
12 public class ExampleSim {
13
14     public static void main(String[] args) {
15
16         Game g = new Game(15,5);
17         int p1Wins=g.play(1000);
18         System.out.println("player 1 wins: "+ p1Wins + " games!");
19
20
21     } // end main
22 } // end class
23
```

```
1  //*****
2  //   Game class for Pig
3  //
4  // This class models the game for Pig
5  // the no-argument constructor has been slightly modified
6  // so that it is compatible with the modifications we made to Player
7  // The play method has been slightly modified to include more of the print
8  // statements previously in the Player class
9  //
10 // You need to add a play method that takes an int parameter indicating
11 // how many games to play and returns the number of games one by p1.
12 //
13 //*****
14
15
16 import java.util.Scanner;
17
18
19 //Cut & Pasted from VS Code debugging work 2/24/2021 8:10 PM EDT --Jinku
20
21 public class Game {
22
23     private Player p1; //human player
24     private Player p2; //computer player
25
26     // This is my default constructor for interactive play
27     public Game(){
28         p1=new Player();
29         p2=new Player(20); // create a computer player with threshold 20
30     }
31     // This is my constructor to use for simulated play
32     public Game(int first, int second){
33         p1=new Player(first);
34         p2=new Player(second);
35     }
36
37     // This is my default play() method
38     public void play() {
39
40         while(p1.getScore()<100 && p2.getScore()<100) {
41             if(p2.getScore() < 100) {
42                 p1.play();
43                 System.out.println("Human player is done");
44                 System.out.println("Human player's total score is: " + p1.getScore() + "\n");
45
46                 if(p1.getScore() < 100){
47                     p2.play();
48                     System.out.println("CPU player is done");
49                     System.out.println("CPU player's total score is: " + p2.getScore() + "\n");
```

```

50     }
51
52     if(p1.getScore() >= 100) {
53         p1.setGamesWon();
54         System.out.println("Human player, you win!" + "\n");
55         break;
56     }
57     else if (p2.getScore() >= 100) {
58         p2.setGamesWon();
59         System.out.println("Human player, you lose!" + "\n");
60         break;
61     }
62 }
63 }
64 // This is my play(int repeats) method for executing the play method for n-repeated simulated games
of Pig
65 public int play (int repeats) {
66
67     int i = 0;
68
69     while(p1.getScore()<100 && p2.getScore()<100 && i < repeats) {
70
71         if(p2.getScore() < 100) {
72             p1.play();
73             System.out.println("CPU1 is done");
74             System.out.println("CPU1 total score is: " + p1.getScore() + "\n");
75
76             if(p1.getScore() < 100) {
77                 p2.play();
78                 System.out.println("CPU2 is done");
79                 System.out.println("CPU2 total score is: " + p2.getScore() + "\n");
80             }
81
82             if(p1.getScore() >= 100) {
83                 p1.setGamesWon();
84                 p1.reset();
85                 p2.reset();
86                 System.out.println("CPU1, you win! \n");
87                 i++;
88             }
89             else if (p2.getScore() >= 100) {
90                 p2.setGamesWon();
91                 p1.reset();
92                 p2.reset();
93                 System.out.println("CPU1, you lose! \n");
94                 i++;
95             }
96         }
97
98     }
99     return p1.getGamesWon();
100 }

```

```
101 }
102
103
104 // this method should play _repeats_ games
105 // and should return the number of games that p1 wins
106
107 // your code here
108
109
110
111 // end of new play method
112
113
114
115 // end class
```

```
1  //*****
2  //   Test class simulating Pig computer vs. computer
3  //
4  // play 1000 games against each other and report
5  // the winning percentage of player 1
6  //
7  //
8  //*****
9
10
11 //Cut & Pasted from VS Code debugging work 2/24/2021 8:10 PM EDT --Jinku
12
13 public class PigSim {
14
15     public static void main(String[] args) {
16         double p1WinPercentage;
17         final int nSims = 1000;
18         final int p1Threshold = 10;
19         final int p2Threshold = 20;
20         double percent = 0;
21
22         Game g = new Game(p1Threshold, p2Threshold);
23         int p1Wins = g.play(nSims);
24         percent += (double) p1Wins / nSims;
25         percent *= 100;
26
27         System.out.println("Player 1 won "+ percent + "% of his/her games!");
28         // your code here
29         // use the ExampleSim.java file I have provided to get started
30         // then add some constants to make it easier to experiment with
31         // different strategies.
32
33     } // end main
34 } // end class
35
36
```

```
1  //*****
2  //   models a Player in Pig
3  // this class has been modified
4  //
5  //   1) a threshold instance variable t has been added
6  //   2) the constructor has been changed
7  //   3) the constructor has been overloaded
8  //   4) the compPlay has fewer print statements
9  //   5) a new reset method has been added to reset score to 0
10 //
11 //
12 //*****
13
14
15 import java.util.Scanner;
16
17 //Cut & Pasted from VS Code debugging work 2/24/2021 8:10 PM EDT --Jinku
18
19 public class Player {
20
21     private boolean isHuman;
22     private int score;
23     private Die myDie;
24     private Scanner input;
25     private int t;
26     private int gamesWon;
27
28     //default constructor will create a human player
29     public Player(){
30         isHuman = true;
31         myDie=new Die(6);
32         score=0;
33         input = new Scanner(System.in);
34         // don't need to initialize t
35
36     }
37
38     public Player(int threshold){
39         isHuman=false;
40         myDie=new Die(6);
41         score=0;
42         t=threshold;
43         gamesWon = 0;
44         // don't need to initialize the Scanner
45     }
46
47
48     public void play() {
49         if(isHuman)
```



```

50     playHuman();
51     else
52         playComp();
53 }
54
55
56 private void playHuman() {
57     int current=0;
58     int runningTotal=0;
59     int again=1;
60
61     while(again==1) {
62         myDie.roll();
63         current=myDie.getSide();
64         System.out.println("You rolled a " + current);
65         if (current == 1) {
66             again=0;
67             runningTotal=0;
68         }
69         else {
70             runningTotal=runningTotal+current;
71             System.out.println("Your round total is now " + runningTotal);
72             System.out.println("Roll again? (1=yes, 0=no)");
73             again=input.nextInt();
74         }
75
76     }//end while
77     score=score+runningTotal;
78     System.out.println("Your total score is: " + score);
79 }//end method
80
81 private void playComp() {
82     int current=0;
83     int runningTotal=0;
84
85     while(runningTotal < t && current != 1 && score + runningTotal < 100) {
86         myDie.roll();
87         current=myDie.getSide();
88         // comment this out since we don't want it for the simulation.
89         // System.out.println("Computer rolled a " + current);
90         if (current == 1) {
91             runningTotal = 0;
92         }
93         else{
94             runningTotal = current+runningTotal;
95         }
96     }//end while
97     score=score+runningTotal;
98
99     System.out.println("***** Final score for the current turn is: " + this.getScore() + "
*****\n");
100 } //end playComp

```

```
101
102 //Accessor Methods
103
104 //This method returns the current number of games won by the player object called.
105
106 public int getGamesWon() {
107     return gamesWon;
108 }
109
110 //This method returns the current score in the current game of the player object called.
111
112 public int getScore() {
113     return score;
114 }
115
116 //Mutator Methods
117
118 //This method resets the score to zero after the said game has been won, unto the player object
called.
119
120 public void reset(){
121     score=0;
122 }
123
124 //This method increments the gamesWon counter variable by 1 each time it is called unto the player
object called.
125
126 public void setGamesWon() {
127     this.gamesWon++;
128 }
129
130
131 }
132 // end class
133
134
135
```

COMS W1004

In-class Exercise 3

In this virtual Codio box you will find today's in-class exercise!

Today we're going to modify the Pig example we just saw with the goal of creating a new test class we can use to test computer strategies. I have made some modifications for you. We will discuss those in class. Here's what you need to do:

1) overload the play method in the `Game` class so that you can choose to play **x** number of simulated games and the method returns the number of times **p1** wins. Your Game class must work the ExampleSim class provided here.

2) Write the main method in the `PigSim` class. This method should create a game with two players one with threshold 10 and the other with threshold 20. Have them play 1000 games against each other and report the **percentage** of times **p1** wins. This is essentially just creating your own test class that looks very similar to the ExampleSim class included here.

3) Modify your PigSim class by adding constants so that you can change the strategies and number of games easily and experiment with different computer strategies.

****Submitting your work:****

Please submit your work electronically on Gradescope. To do this, export your .java files from Codio, compress them into a single zip file; please name this file [your UNI here]_ex3.zip (without the brackets). So I would submit a single file named ac1076_ex3.zip

Submit this file under **In-Class Exercise 3**. *****Remember, In-class exercises may NOT be submitted late!*****